

SepReformer

Kiến trúc của SepReformer được thiết kế dựa trên **khung mã hóa-giải mã bất đối xứng (Asymmetric Encoder-Decoder)**

Sơ đồ tổng quan của mô hình:

```
Input → AudioEncoder → [SepEncStage (Global/Local)] → SpkSplitStage  
→ Decoder (Shared + Cross-Speaker) → OutputLayer → AudioDecoder  
→ Separated Speech
```

1. Encoder (Mã hóa)

- **Mục đích :**

- **Tiền xử lý:** Chuẩn bị dữ liệu cho quá trình tách biệt.
- **Phân tích đa mức độ:** Kết hợp cả ngữ cảnh toàn cục và cục bộ.
- **Hỗ trợ kiến trúc bất đối xứng:** Tạo điều kiện cho việc tách sớm và tái tạo hiệu quả.
- **Tối ưu hóa tính toán:** Giảm độ phức tạp thông qua downsampling và cơ chế attention hiệu quả.

- **Thành phần :**

- **AudioEncoder(Lớp mã hóa cơ bản):**

- Sử dụng `Conv1d` với kernel size nhỏ (ví dụ: 3) và GELU activation để mã hóa tín hiệu đầu vào thành đặc trưng.
- Đầu vào: Tín hiệu âm thanh thô dạng `[B, T]` (B: batch size, T: số mẫu thời gian).
- Đầu ra: Đặc trưng dạng `[B, F, T']` (F: số kênh, T': độ dài rút gọn).

```
class AudioEncoder(torch.nn.Module):  
    def __init__(self, in_channels: int, out_channels: int, kernel_size: int, stride: i
```

```

super().__init__()
self.conv1d = torch.nn.Conv1d(
    in_channels=in_channels,
    out_channels=out_channels,
    kernel_size=kernel_size,
    stride=stride,
    groups=groups,
    bias=bias
)
self.gelu = torch.nn.GELU()

def forward(self, x: torch.Tensor):
    if x.dim() == 2: # Nếu input dạng [B, T], thêm channel dimension
        x = x.unsqueeze(1) # [B, 1, T]
    x = self.conv1d(x) # Convolution
    x = self.gelu(x) # Activation
    return x # [B, F, T']

```

◦ **SepEncStage (Các tầng encoder) :**

- **Downsampling** : Giảm độ dài chuỗi thời gian 2 lần qua mỗi tầng bằng **depthwise convolution** (kernel size 5, stride=2).
- **Xử lý Global/Local Transformer** :
 - **GlobalBlock** : Sử dụng **Efficient Gated Attention (EGA)** để nắm bắt ngữ cảnh toàn cục, kết hợp downsampling để giảm độ phức tạp.
 - **LocalBlock** : Sử dụng **Convolutional Local Attention (CLA)** với convolution 1D để xử lý pattern cục bộ.

```

class SepEncStage(torch.nn.Module):
    def __init__(self, in_channels: int, samp_kernel_size: int, **kwargs):
        super().__init__()
        # Downsampling bằng depthwise convolution (stride=2)
        self.down_conv = torch.nn.Conv1d(
            in_channels=in_channels,
            out_channels=in_channels,

```

```

        kernel_size=samp_kernel_size, # Kernel size 5
        stride=2, # Giảm T 2 lần
        padding=(samp_kernel_size - 1) // 2,
        groups=in_channels # Depthwise convolution
    )
    self.BN = torch.nn.BatchNorm1d(num_features=in_channels)
    self.gelu = torch.nn.GELU()
    # Global và Local Transformer
    self.g_block_1 = GlobalBlock(**global_blocks)
    self.l_block_1 = LocalBlock(**local_blocks)
    self.g_block_2 = GlobalBlock(**global_blocks)

def forward(self, x: torch.Tensor):
    # Chuyển shape để áp dụng convolution
    x = x.permute([0, 2, 1]) # [B, T, F] → [B, F, T]
    x = self.down_conv(x) # Downsampling
    x = self.BN(x)
    x = self.gelu(x)
    x = x.permute([0, 2, 1]) # Trả về [B, T, F]
    # Xử lý bằng Global và Local Transformer
    x = self.g_block_1(x, pos_k) # Global Attention
    x = self.l_block_1(x) # Local Attention
    x = self.g_block_2(x, pos_k) # Global Attention
    return x

```

◦ **SpkSplitStage :**

- Tách đặc trưng thành các chuỗi riêng cho từng người nói **ngay sau encoder** (early split) bằng GLU (Gated Linear Unit).
- Đầu ra: **[B, T, num_spks, F]** (num_spks: số người nói).
- **Ưu điểm :**
 - Giảm "nghẽn cổ chai" thông tin (encoder chỉ xử lý 1 chuỗi chung).
 - Decoder tập trung vào phân biệt đặc trưng giữa các speaker.

```

class SpkSplitStage(torch.nn.Module):
    def __init__(self, in_channels: int, num_spks: int):
        super().__init__()
        # Tách đặc trưng bằng GLU
        self.split = torch.nn.Sequential(
            torch.nn.Linear(in_channels, 2 * in_channels * num_spks),
            torch.nn.GLU(dim=-1) # Gated Linear Unit
        )
        self.layer_norm = torch.nn.LayerNorm(in_channels)

    def forward(self, x: torch.Tensor):
        B, T, F = x.shape
        x = self.split(x) # [B, T, F] → [B, T, 2*F*num_spks]
        x = x.view(B, T, num_spks, F) # Tách thành num_spks chuỗi
        x = self.layer_norm(x) # Chuẩn hóa
        return x # [B, T, num_spks, F]

```

2. Decoder (Giải mã)

- **Mục đích** : Tái tạo tín hiệu âm thanh riêng biệt cho từng người nói từ các đặc trưng đã tách.
 - **Phân biệt đặc trưng** của từng người nói thông qua weight-sharing và CS Transformer.
 - **Tối ưu hóa tính toán** bằng cách kết hợp Global/Local Transformer thay thế dual-path.
 - **Tận dụng thông tin đa mức độ** từ encoder qua skip connection để cải thiện độ chính xác.
- **Thành phần** :
 - **Reconstruction Decoder (Các tầng giải mã)** :
 - **Upsampling** : Khôi phục độ dài chuỗi thời gian qua các tầng decoder bằng transposed convolution.

- Sử dụng **shared decoder** (giải mã dùng chung) để phân biệt đặc trưng giữa các speaker.
- **Skip connections** : Kết hợp thông tin từ Encoder để giữ chi tiết.

```

class Separator(torch.nn.Module):
    def __init__(self, num_stages: int, dec_stage: dict, **kwargs):
        super().__init__()
        # Temporal Expanding Part (Decoder stages)
        self.dec_stages = torch.nn.ModuleList([])
        for _ in range(num_stages):
            self.dec_stages.append(SepDecStage(**dec_stage)) # Tầng giải mã

class SepDecStage(torch.nn.Module):
    def __init__(self, in_channels: int, samp_kernel_size: int, **kwargs):
        super().__init__()
        # Upsampling bằng transposed convolution
        self.up_conv = torch.nn.ConvTranspose1d(
            in_channels=in_channels,
            out_channels=in_channels,
            kernel_size=samp_kernel_size, # Kernel size 5
            stride=2, # Khôi phục độ dài T 2 lần
            padding=(samp_kernel_size - 1) // 2,
            groups=in_channels # Depthwise convolution
        )
        self.BN = torch.nn.BatchNorm1d(num_features=in_channels)
        self.gelu = torch.nn.GELU()
        # Global và Local Transformer
        self.g_block_1 = GlobalBlock(**global_blocks)
        self.l_block_1 = LocalBlock(**local_blocks)
        self.g_block_2 = GlobalBlock(**global_blocks)

    def forward(self, x: torch.Tensor, skip: torch.Tensor):
        # Kết hợp skip connection từ Encoder
        x = x + skip # [B, T, F]
        # Upsampling

```

```

x = x.permute([0, 2, 1]) # [B, F, T]
x = self.up_conv(x)      # Khôi phục T 2 lần
x = self.BN(x)
x = self.gelu(x)
x = x.permute([0, 2, 1]) # [B, T, F]
# Xử lý bằng Global và Local Transformer
x = self.g_block_1(x, pos_k)
x = self.l_block_1(x)
x = self.g_block_2(x, pos_k)
return x

```

◦ **Cross-Speaker Attention (CS) :**

- Cho phép các chuỗi đã tách tương tác lẫn nhau để phục hồi thông tin bị mất do **early split**.
- Sử dụng **multi-head self-attention** trên chiều speaker

```

class SpkAttention(torch.nn.Module):
    def __init__(self, in_channels: int, num_mha_heads: int, dropout_rate: float):
        super().__init__()
        self.self_attn = MultiHeadAttention(
            n_head=num_mha_heads,
            in_channels=in_channels,
            dropout_rate=dropout_rate
        )
        self.feed_forward = GCFN(in_channels, dropout_rate)

    def forward(self, x: torch.Tensor, num_spk: int):
        B, T, F = x.shape
        x = x.view(B, T, num_spk, F) # Tách thành [B, T, J, F]
        # Self-attention trên chiều speaker (J)
        x = self.self_attn(x) # [B, T, J, F]
        x = self.feed_forward(x)
        x = x.view(B, T, num_spk * F) # Trả về [B, T, J*F]
        return x

```

- **OutputLayer và AudioDecoder :**

- Tạo mask hoặc tái tạo trực tiếp tín hiệu từ đặc trưng.
- Sử dụng **GLU activation** để điều chỉnh thông tin.

```
class OutputLayer(torch.nn.Module):
    def __init__(self, in_channels: int, out_channels: int):
        super().__init__()
        self.fc = torch.nn.Sequential(
            torch.nn.Linear(in_channels, 2 * out_channels),
            torch.nn.GLU(dim=-1) # Gated activation
        )

    def forward(self, x: torch.Tensor):
        return self.fc(x) # [B, T, out_channels]

class AudioDecoder(torch.nn.Module):
    def __init__(self, in_channels: int, out_channels: int, kernel_size: int, stride: int):
        super().__init__()
        self.deconv = torch.nn.ConvTranspose1d(
            in_channels=in_channels,
            out_channels=out_channels,
            kernel_size=kernel_size,
            stride=stride
        )

    def forward(self, x: torch.Tensor):
        x = x.permute([0, 2, 1]) # [B, F, T]
        x = self.deconv(x) # Khôi phục về độ dài ban đầu
        return x.permute([0, 2, 1]) # [B, T, 1]
```

3. Khối Transformer Toàn cục và Cục bộ

- **GlobalBlock (Efficient Gated Attention - EGA) :**

- **Self-attention** với **relative positional encoding** để xử lý ngữ cảnh dài.

- **Downsampling** chuỗi trước khi áp dụng attention để giảm độ phức tạp.
- **Gating mechanism** (sigmoid) để điều chỉnh thông tin.

```
class GlobalBlock(torch.nn.Module):
    def __init__(self, in_channels: int, num_heads: int, dropout_rate: float, maxlen: int):
        super().__init__()
        self.ega = EGA(in_channels, num_heads, dropout_rate, maxlen) # Efficient Global Attention
        self.gcfn = GCFN(in_channels, dropout_rate) # Gated Convolutional Feed-Forward

    def forward(self, x: torch.Tensor, pos_k: torch.Tensor):
        x = self.ega(x, pos_k) # Self-attention với positional encoding
        x = self.gcfn(x) # Feed-forward
        return x
```

- **LocalBlock (Convolutional Local Attention - CLA) :**

- **Self-attention cục bộ** với cửa sổ trượt, kết hợp convolution 1D để nắm bắt pattern thời gian ngắn.

```
class LocalBlock(torch.nn.Module):
    def __init__(self, in_channels: int, kernel_size: int, dropout_rate: float):
        super().__init__()
        self.cla = CLA(in_channels, kernel_size, dropout_rate) # Local Attention
        self.gcfn = GCFN(in_channels, dropout_rate) # Feed-forward

    def forward(self, x: torch.Tensor):
        x = self.cla(x) # Self-attention cục bộ
        x = self.gcfn(x)
        return x
```

4. Cơ chế "Early Split and Shared Decoder" (ESSD)

- **Early Split** : Tách đặc trưng thành các speaker **sau encoder** , tránh nghẽn cổ chai thông tin.

- **Shared Decoder** : Decoder dùng chung trọng số cho tất cả speaker, kết hợp **cross-speaker attention** để tương tác giữa các chuỗi.
-

5. U-Net Structure

- **Skip Connections** : Kết nối các tầng encoder và decoder để giữ thông tin chi tiết từ đầu vào.
 - **Xử lý nhiều stage** : Xử lý đặc trưng ở nhiều độ phân giải thời gian khác nhau (từ encoder stages đến decoder stages).
-

6. Multi-Loss Training

- **Auxiliary Loss** : Áp dụng loss ở nhiều tầng decoder để tăng cường học phân biệt giữa các speaker.
- **Cross-Speaker Interaction** : Kết hợp thông tin giữa các chuỗi đã tách qua cơ chế attention.